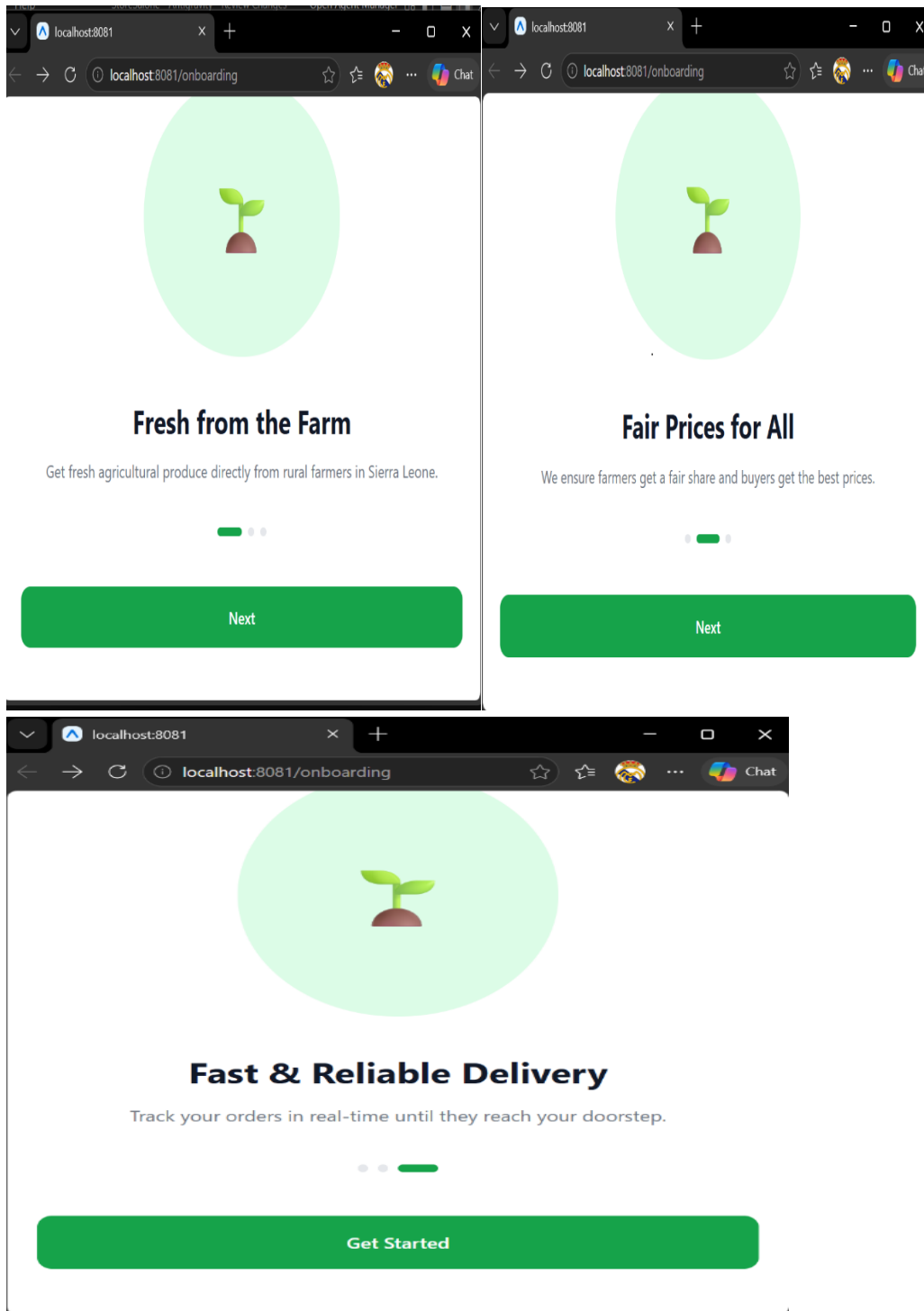


SaloneAgro Store



1. Introduction

1.1 Overview of SaloneAgro Store

SaloneAgro Store is a cross-platform mobile commerce application built to digitally connect rural farmers in Sierra Leone with urban and peri-urban buyers, while also coordinating the motorcycle delivery riders and platform administrators who keep the marketplace running. The application removes the layers of middlemen that typically sit between a farmer's harvest and a buyer's table, allowing farmers to list and price their own produce, buyers to order with confidence, and the platform itself to monitor quality, fairness, and delivery performance across the country.

The system was designed and built as a production-grade mobile commerce platform using React Native with Expo for the client application, Firebase Authentication for identity management, Supabase (PostgreSQL) for the application database and real-time data layer, and Stripe combined with local Mobile Money rails (Orange Money and Afrimoney) for payment processing. Together these technologies allow SaloneAgro Store to run natively on iOS and Android while also serving a fully functional web build from the same codebase.

Unlike a simple two-sided marketplace, SaloneAgro Store supports four distinct user roles — Buyer, Farmer, Driver, and Admin — each with its own dedicated dashboard, navigation flow, and permission set. This makes the application closer in spirit to a logistics-aware commerce platform than a basic storefront, since it must track not only what is bought and sold, but who delivers it, how it is paid for, and how the platform is governed.

1.2 Purpose and Scope of Documentation

This documentation provides a complete technical and functional description of SaloneAgro Store as it stands at Version 1.0.0. It is intended to serve three audiences at once: developers who need to understand how the system is built and how to extend it, evaluators and reviewers who need to assess the system against a defined set of requirements, and stakeholders who want a plain description of what the application does and how a person would actually use it.

The scope of this document follows eight major areas: the system's architecture and how data flows between its parts; the functional and non-functional requirements the system was designed to satisfy; the technologies and configuration used to implement it; the day-to-day operational procedures for running and maintaining it; the security and compliance measures protecting user data and payments; the testing and validation work carried out before release; and finally a summary of findings together with recommendations for future improvement.

Each functional area of the application — from the authentication screens through to the admin transaction monitor — is described in narrative form, screen by screen, so that a reader unfamiliar with the codebase can still follow exactly what happens at each step of a buyer's, farmer's, driver's, or administrator's journey through the app.

1.3 Project Background

SaloneAgro Store addresses a long-standing structural problem in Sierra Leone's agricultural economy: farmers in rural districts such as Makeni, Bo, Kenema, and Kabala frequently have no direct channel to urban buyers in Freetown and other commercial centres. Produce typically passes through several intermediaries before reaching a consumer, with each link in the chain taking a margin. The result is that farmers receive a smaller share of the final sale price than the quality of their produce would justify, while buyers pay more than necessary for goods that have aged in transit through multiple hands.

By building a mobile-first digital marketplace with an integrated delivery network, SaloneAgro Store gives farmers a direct storefront, gives buyers transparent pricing and order tracking, and gives the platform operator the tools needed to verify sellers, monitor transactions, and keep deliveries moving reliably across the country.

1.4 Project Objectives

The system was designed to meet the following objectives:

- Provide a direct, disintermediated channel between farmers and buyers, so that pricing set by the farmer reaches the buyer without unnecessary markup.
- Support local, real-world payment methods — including card payments through Stripe and Mobile Money payments through Orange Money and Afrimoney — alongside cash on delivery.
- Coordinate a motorcycle delivery network with real-time tracking, so that buyers can see exactly where their order is between confirmation and doorstep delivery.
- Offer role-specific dashboards for Buyers, Farmers, Drivers, and Admins, each tailored to the tasks that role actually performs.
- Apply a consistent, modern UI/UX design system across all four roles while still giving each role its own visual identity and information priorities.
- Build on a scalable cloud backend (Firebase for identity, Supabase for data) capable of supporting real-time order and delivery updates as usage grows.

2. System Architecture

2.1 Components and Data Flow

SaloneAgro Store follows a three-tier architecture consisting of a Mobile and Web Client Layer, a Backend Services layer, and a set of External Services that the backend integrates with on the client's behalf. The client layer is a single React Native (Expo) codebase that compiles to native iOS and Android builds as well as a static web build, so the same screens, components, and business logic run everywhere with only minimal platform-specific branching (for example, the Stripe wrapper and the map component each have a dedicated web fallback).

Inside the client layer, two lightweight Zustand stores — `useAuthStore` and `useCartStore` — hold the global state that every screen needs immediate access to: who is logged in, what role they hold, and what is currently sitting in their cart. Everything else that comes from the server — product listings, order records, transaction history — is fetched and cached through TanStack React Query, which handles background refetching so that screens like the buyer's order list or the admin transaction monitor stay reasonably fresh without the user having to manually refresh.

The backend layer is deliberately split across two providers rather than one. Firebase Authentication is responsible exclusively for identity: registering accounts, verifying email and password credentials, and issuing the session that the rest of the app treats as proof of who the user is. Supabase, a hosted PostgreSQL service, is responsible for everything else — the products table, the orders table, the users and transactions tables, and the real-time subscriptions that push order-status and driver-location updates to subscribed clients the moment a row changes.

Sitting alongside the database is the payments layer, which is itself split between Stripe (for card payments, via the native Stripe Payment Sheet on mobile and the Stripe.js Elements provider on web) and the Mobile Money APIs for Orange Money and Afrimoney. A fourth payment path, cash on delivery, requires no external integration at all and is handled entirely within the order record.

Finally, React Native Maps (with a web-compatible fallback component) connects to the driver-location real-time channel to render the live tracking view that buyers and admins use to follow a delivery from pickup to doorstep.

2.1.1 High-Level Architecture Diagram

The diagram below summarises how the four layers connect. The client layer (mobile app and web app) reads and writes through the two Zustand stores, which in turn talk to the backend services. Firebase Authentication connects out to the Google OAuth provider for social sign-in, while the Stripe gateway connects out to the Orange Money and Afrimoney APIs to settle

Mobile Money transactions. React Native Maps draws data from the same backend layer to plot delivery routes.

Layer	Component	Connects To
Client	Mobile App (iOS/Android)	Zustand state stores
Client	Web App (Browser)	Zustand state stores
State	useAuthStore	Firebase Authentication
State	useCartStore	Supabase (orders on checkout)
Backend	Firebase Authentication	Google OAuth Provider
Backend	Supabase (Database / Storage)	React Query cache layer
Backend	Stripe Payment Gateway	Orange Money API, Afrimoney API
External	React Native Maps	Supabase real-time driver-location channel

2.1.2 Data Flow Summary

A typical buyer session illustrates how these layers cooperate. On launch, the client checks useAuthStore for a persisted session; if one exists, it is verified against Firebase and the user is routed straight to their role-based dashboard. As the buyer browses the marketplace, React Query fetches the product catalogue from Supabase and caches it locally, refetching quietly in the background. When the buyer adds an item to their cart, useCartStore updates immediately and entirely on-device — no network round-trip is required until checkout. At checkout, the cart contents and delivery details are sent to Supabase to create an order record, the chosen payment method routes the transaction through Stripe or the relevant Mobile Money API, and once payment is confirmed, Supabase's real-time channel begins streaming order-status and driver-location updates back to the buyer's tracking screen.

2.2 Integration with External Services

SaloneAgro Store integrates with four categories of external service, each chosen to satisfy a specific functional requirement of a Sierra Leone-focused commerce platform.

External Service	Purpose	Integration Point
Google OAuth Provider	Social sign-in as an alternative to email/password	Firebase Authentication (web)
Stripe	Card payment processing for buyers who prefer card over Mobile Money	checkout/payment.tsx (native + web)

External Service	Purpose	Integration Point
Orange Money API	Mobile Money payment authorization and settlement	Mobile Money payment flow
Afrimoney API	Mobile Money payment authorization and settlement	Mobile Money payment flow
React Native Maps	Live map rendering for delivery tracking	tracking/index.tsx

2.3 Architecture Pattern

The application follows a Feature-Based Modular Architecture, meaning code is organised primarily around what it does (authentication, cart, checkout, tracking) rather than around technical layer alone. This keeps each feature's screens, state, and supporting components close together, which matters in a four-role application where a change to, say, the Farmer dashboard should not risk touching Buyer or Driver code paths.

Layer	Technology	Responsibility
Presentation	React Native + NativeWind	UI rendering, user interactions, animations
Navigation	Expo Router (file-based)	Screen routing, deep linking, role-based navigation
State	Zustand	Global state management for auth and cart
Data	TanStack React Query	Server state caching, background refetching
Backend	Firebase + Supabase	Authentication, database, real-time subscriptions
Payments	Stripe + Mobile Money	Payment processing and settlement

3.1 Functional Requirements

The functional requirements below describe what each of the four user roles — Buyer, Farmer, Driver, and Admin — must be able to do within SaloneAgro Store. These requirements were used as the basis for the screens and modules described later in Section 4 and Section 5.

3.1.1 Buyer Requirements

- Register and log in using either an email and password or a Google account.
- Browse a categorised, searchable catalogue of farm produce with images, pricing, and farmer attribution.
- Add products to a persistent shopping cart, adjust quantities, and remove items.

-
- Complete checkout by entering delivery details and choosing from four payment methods: card, Orange Money, Afrimoney, or cash on delivery.
 - Track an active delivery in real time on a map, including driver identity and estimated arrival.
 - View a complete order history with status badges and reorder past purchases.
 - Manage account settings, including profile details and logout.

3.1.2 Farmer Requirements

- Register with the Farmer role and access a dedicated farmer dashboard distinct from the buyer marketplace.
- Create, edit, and remove product listings, including an image, price, unit of measurement, and description.
- View monthly earnings, active listing counts, and total order volume at a glance.
- Receive and manage incoming orders, updating their status as they are fulfilled.

3.1.3 Driver Requirements

- Toggle an online/offline availability status to control whether new deliveries are assigned.
- View an active delivery, including pickup and drop-off locations, with a one-tap navigation option.
- Review a complete delivery history and a running record of earnings and payouts.

3.1.4 Admin Requirements

- View platform-wide key metrics, including total registered users and active verified farmers.
- Review and approve or reject pending farmer verification applications.
- Manage all registered user accounts across every role.
- Monitor every order placed on the platform, regardless of which farmer or buyer is involved.
- Monitor payment transactions and process refunds where required.

3.1.5 Cross-Role Requirement Matrix

The table below consolidates the functional requirements into a single role-permission matrix, making it straightforward to confirm that no role has access to a capability outside its intended scope.

Feature	Buyer	Farmer	Driver	Admin
Browse Products	Yes	No	No	No
Place Orders	Yes	No	No	No
Shopping Cart	Yes	No	No	No
Track Delivery	Yes	No	No	Yes
View Order History	Yes	Yes	Yes	Yes
Add Product Listings	No	Yes	No	No
View Earnings	No	Yes	Yes	No
Accept Deliveries	No	No	Yes	No
Online/Offline Toggle	No	No	Yes	No
Verify Farmers	No	No	No	Yes
Manage Users	No	No	No	Yes
Monitor Transactions	No	No	No	Yes
System-wide Orders	No	No	No	Yes
Account Settings	Yes	Yes	Yes	Yes

3.2 Non-Functional Requirements

Beyond what the system must do, SaloneAgro Store was designed against a set of quality attributes that determine how well it does it.

3.2.1 Performance

- Product catalogue and order data are cached client-side through React Query, so previously loaded screens render instantly while fresher data is fetched in the background.
- Real-time order-status and driver-location updates are delivered through Supabase's real-time channel rather than manual polling, keeping update latency low without placing constant load on the client.

3.2.2 Usability

- A single, consistent design language (the SaloneAgro colour and component system) is applied across all four roles, while each role retains its own accent colour so users always know which mode of the app they are in.
- A three-step onboarding carousel introduces first-time users to the platform's value proposition before they reach the login screen.

3.2.3 Security

- All authentication is delegated to Firebase Authentication rather than custom credential storage.
- Card details are never stored by the application itself; all card data is handled directly by the Stripe Payment Sheet / Elements provider.

3.2.4 Portability

- A single Expo codebase targets iOS, Android, and web from the same source, with platform-specific files (for example, Map.web.tsx and StripeWrapper.web.tsx) used only where native APIs are unavailable in the browser.

3.2.5 Localisation and Market Fit

- All monetary values are presented in Sierra Leonean Leone (Le).
- Supported payment methods are deliberately matched to the methods most commonly used in Sierra Leone: Orange Money, Afrimoney, card, and cash on delivery.

4.1 Technologies Used

SaloneAgro Store is built almost entirely on a JavaScript and TypeScript toolchain, chosen to allow a single team to cover frontend, backend integration, and payment logic without context-switching between languages.

4.1.1 Core Application Framework

Package	Version	Purpose
expo	~54.0.34	Application framework and build toolchain
react	19.1.0	UI component library
react-native	0.81.5	Native platform bridge
expo-router	~6.0.23	File-based navigation system
typescript	~5.9.2	Static type checking

4.1.2 Backend and Services

Package	Version	Purpose
firebase	^12.14.0	Authentication (Email/Password, Google OAuth)
@supabase/supabase-js	^2.108.1	PostgreSQL database, real-time subscriptions, storage

Package	Version	Purpose
@stripe/stripe-react-native	0.50.3	Native payment sheet integration
@stripe/stripe-js	^9.8.0	Web payment processing

4.1.3 UI and Styling

Package	Version	Purpose
nativewind	^4.2.5	Tailwind CSS for React Native
tailwindcss	^3.4.19	Utility-first CSS engine
lucide-react-native	^1.18.0	Icon library
react-native-reanimated	~4.1.1	Performant animations
@gorhom/bottom-sheet	^5.2.14	Bottom sheet component
react-native-maps	1.20.1	Map rendering for delivery tracking

4.1.4 State and Data

Package	Version	Purpose
zustand	^5.0.14	Lightweight global state management
@tanstack/react-query	^5.101.0	Async data fetching and caching
@react-native-async-storage/async-storage	2.2.0	Persistent local storage

4.2 Configuration and Setup

4.2.1 Project Directory Layout

The codebase is organised so that Expo Router's file-based routing maps directly onto the four user roles, with shared logic and components kept separately under `src/`.

Path	Contents
app/(auth)/	login.tsx, register.tsx, otp.tsx — authentication screens
app/(tabs)/	Buyer tab navigator: home, search, cart, orders, profile
app/(admin)/	Admin dashboard: overview, farmers, users, orders, transactions
app/(farmer)/	Farmer dashboard: overview, add-product, orders
app/(driver)/	Driver dashboard: overview, deliveries, earnings
app/checkout/	Delivery details, payment method, success screens

Path	Contents
app/tracking/	Real-time map view with driver location
src/components/ui/	Reusable primitives: Button, Card, Input
src/components/features/	ProductCard, Map (native), Map.web (web fallback)
src/lib/	firebase.ts and supabase.ts client initialisation
src/store/	useAuthStore.ts and useCartStore.ts (Zustand)
src/theme/	colors.ts — design system colour tokens

4.2.2 Environment Configuration

Three environment variables are required to connect the client to its backend services, all of which are exposed to the Expo client via the EXPO_PUBLIC_ prefix:

Variable	Description	Required
EXPO_PUBLIC_SUPABASE_URL	Supabase project REST URL	Yes
EXPO_PUBLIC_SUPABASE_ANON_KEY	Supabase public/anon key	Yes
EXPO_PUBLIC_STRIPE_PUBLISHABLE_KEY	Stripe publishable key	Yes

4.2.3 Firebase Configuration

Firebase Authentication is initialised against the ffarm-9b2c1 project, which is responsible solely for verifying user identity. No application data — products, orders, or transactions — is stored in Firebase; that responsibility belongs entirely to Supabase, described in Section 5.

4.2.4 Application Manifest (app.json)

Setting	Value
App Name	StoreSalone
Slug	StoreSalone
Version	1.0.0
Orientation	Portrait
URL Scheme	storesalone://
New Architecture	Enabled
Typed Routes	Enabled
React Compiler	Enabled
Web Output	Static

4.2.5 Design System

A single colour-token file (`src/theme/colors.ts`) drives the visual identity of the entire application, ensuring that Buyer, Farmer, Driver, and Admin surfaces all draw from the same palette even as each role applies its own accent.

Token	Hex	Usage
primary	#16a34a	CTA buttons, active states, success indicators
primaryDark	#15803d	Pressed / hover states
earth	#a16207	Pricing, farmer dashboard accent
background	#f3f4f6	Screen backgrounds
surface	#ffffff	Cards and panels
text	#111827	Primary text
textMuted	#6b7280	Secondary text
border	#e5e7eb	Dividers and borders
error	#ef4444	Error states, destructive actions
success	#10b981	Success states

4.2.6 Root Provider Hierarchy

Every screen in the application is rendered inside a fixed stack of providers established in the root layout file. From outermost to innermost, this is: `StripeProvider`, wrapping `GestureHandlerRootView`, wrapping `QueryClientProvider` (React Query), wrapping `ThemeProvider` (light/dark mode support), wrapping the root `Stack Navigator`. This ordering ensures payments, gesture handling, server-state caching, and theming are all available to every screen regardless of which role's route group is active.

4.3 Application Screens and Features

This section walks through every screen in SaloneAgro Store in the order a user actually encounters them, describing what each screen shows and what a person can do on it. The walkthrough is organised first around the screens every user passes through (onboarding and authentication), then around each of the four role-specific dashboards in turn.

4.3.1 Onboarding Screen

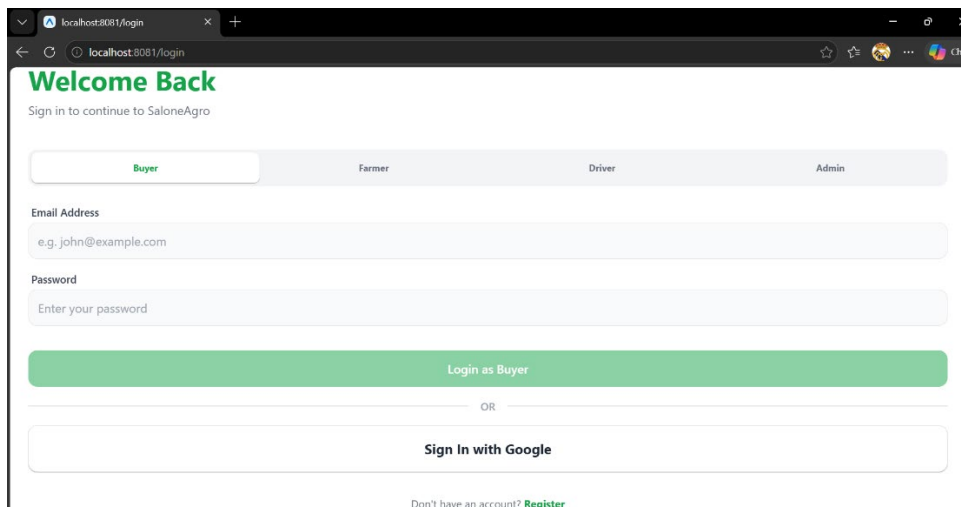
On first launch, before any login screen appears, a new user is shown a three-step onboarding carousel introducing the platform's purpose. The first slide, "Fresh from the Farm," explains that the app connects buyers directly with rural farmers across Sierra Leone. The second, "Fair Prices for All," emphasises that farmers receive a fair share of the sale price while buyers still

get competitive pricing. The third, “Fast & Reliable Delivery,” introduces the real-time tracking feature that follows an order until it reaches the buyer’s doorstep. Once the carousel is dismissed, the user proceeds to the Login screen, and the onboarding flow does not reappear on subsequent launches.

4.3.2 Login Screen

The Login screen offers two ways to sign in, both available on every platform the app runs on: email and password authentication via Firebase’s `signInWithEmailAndPassword` method, and Google OAuth sign-in (currently available on the web build through Firebase’s `signInWithPopup` and `GoogleAuthProvider`). The screen validates that an email and password have been entered before attempting authentication, and displays a clear error message if Firebase rejects the credentials. On a successful login, the user is routed automatically to the dashboard that matches their stored role — Buyer to the tab navigator, Farmer to the farmer dashboard, Driver to the driver dashboard, or Admin to the admin dashboard — with no further action required from the user.

Diagram of Login Screen



4.3.3 Register Screen

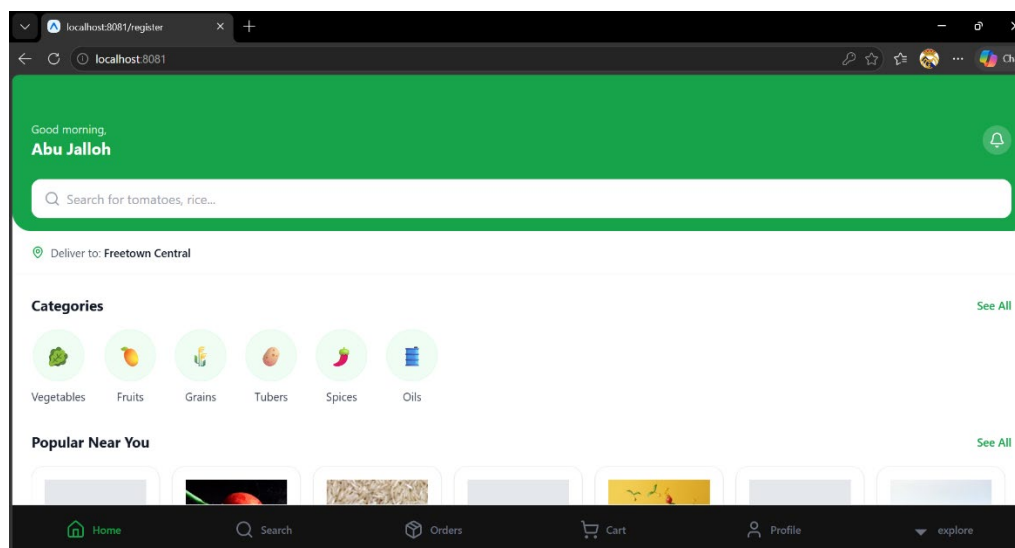
New users create an account on the Register screen by supplying a full name, email address, phone number, password, and a selected role of Buyer, Farmer, or Driver. Input filtering is applied as the user types: the name field strips any character that is not a letter or space, and the phone field strips any character that is not a digit or a leading plus sign, reducing the chance of malformed data reaching Firebase. The email field uses the device’s email-optimised keyboard, and the password field is masked using `secureTextEntry`. Once the form passes validation, `createUserWithEmailAndPassword` creates the Firebase account, the chosen role is stored, and the user is routed directly into the dashboard for that role.

Diagram of Register Screen

4.3.4 Buyer Home Screen (Marketplace)

The Home tab is the buyer's entry point into the marketplace. It opens with a personalised greeting, followed by a search bar that filters the product grid in real time as the buyer types. Below the search bar sits a row of category filters, allowing the buyer to narrow the catalogue to Vegetables, Fruits, Grains, Tubers, Spices, or Oils with a single tap. The bulk of the screen is a responsive product grid, with each card displaying the product image, name, price in Leone, the supplying farmer or cooperative, and an Add to Cart action that updates the cart immediately without leaving the screen.

Diagram of Buyer Dashboard



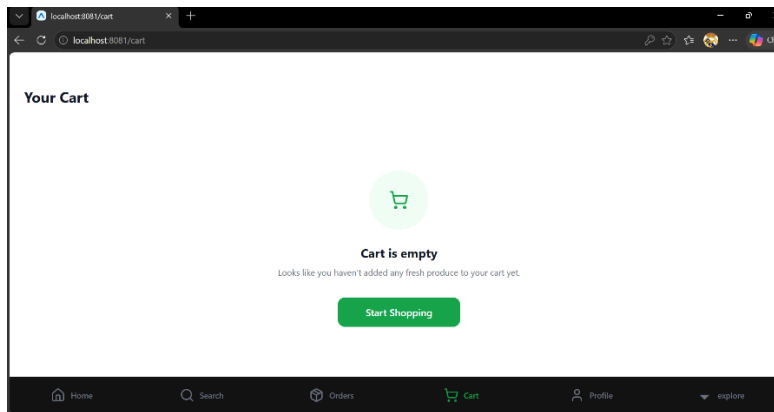
4.3.6 Search Screen

The Search tab provides a dedicated full-text search experience over the same product catalogue shown on the Home screen, combined with the same category filters. This gives buyers a focused way to look for a specific item by name when they already know what they want, rather than browsing the general marketplace grid.

4.3.7 Cart Screen

The Cart tab lists every item the buyer has added, each shown with its thumbnail, unit price, current quantity, and line-item subtotal. Quantity controls allow the buyer to increase or decrease how many units of an item they want directly from this screen, with the cart automatically removing an item if its quantity is reduced to zero. A running total is calculated across all items using the cart store's `getTotal` function, and a fixed delivery fee of Le 15,000 is applied per order. A Proceed to Checkout button at the bottom of the screen moves the buyer into the checkout flow once they are satisfied with their selection.

Diagram of Cart Screen

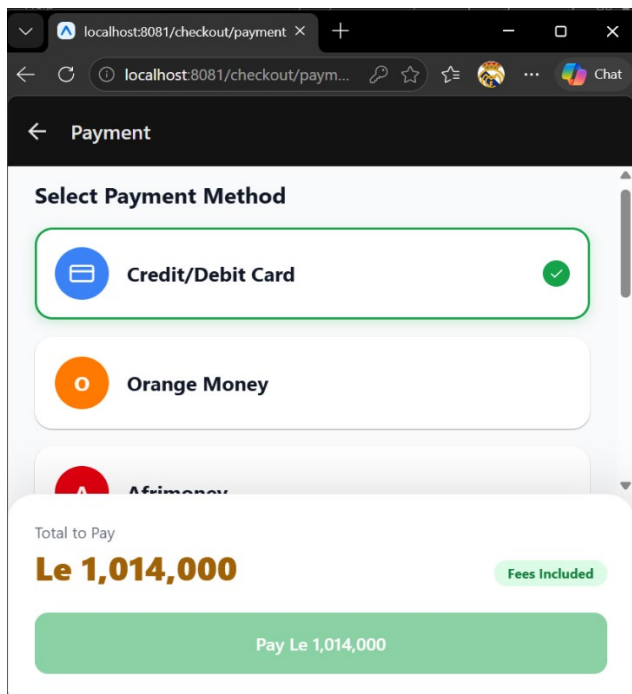


4.3.8 Checkout Flow

Checkout is presented as a three-step linear flow: Delivery Details, Payment Method, and a final Success screen, with a processing overlay shown between the second and third steps while the selected payment method is confirmed.

In the first step, Delivery Details, the buyer enters a delivery address (required) and an optional delivery note, alongside a summary of the order they are about to place. In the second step, Payment, the buyer chooses from four supported payment methods — card, Orange Money, Afrimoney, or cash on delivery — and enters whatever details that method requires before confirming. A brief processing overlay is shown while the payment is authorised. In the third step, Success, the buyer sees an animated confirmation that the order has been placed, the cart is automatically cleared, and the new order appears immediately on the buyer's Orders screen.

Diagram of Checkout Flow

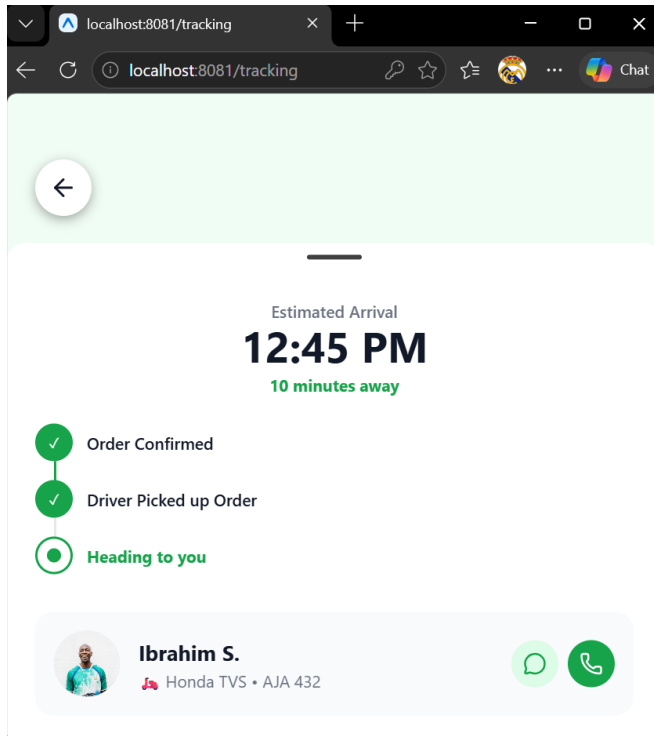


4.3.9 Orders Screen (Active and History)

The Orders tab is split into Active and History views. The Active view shows orders that have not yet reached Completed status, each carrying a status badge (Pending Pickup, Processing, or in transit) that updates live as the order progresses. The History view shows past orders that have reached a final state, whether Completed or Cancelled, and allows the buyer to review what was ordered, when, and for how much. Buyers can tap into any active order to open the real-time Tracking screen described below.

4.3.10 Real-Time Delivery Tracking Screen

The Tracking screen renders a live map — using react-native-maps natively, with a web-compatible fallback component on the browser build — showing the assigned driver's current marker alongside the destination pin, plotted around the relevant Sierra Leone coordinates for the order. Above or alongside the map, a progress tracker steps the buyer through three stages: Order Confirmed, Driver Picked Up, and Heading to You. A driver info panel, presented using a bottom sheet component, surfaces the driver's name, vehicle details (for example, "Honda TVS • AJA 432"), and direct call and message buttons, alongside a prominent estimated time of arrival.



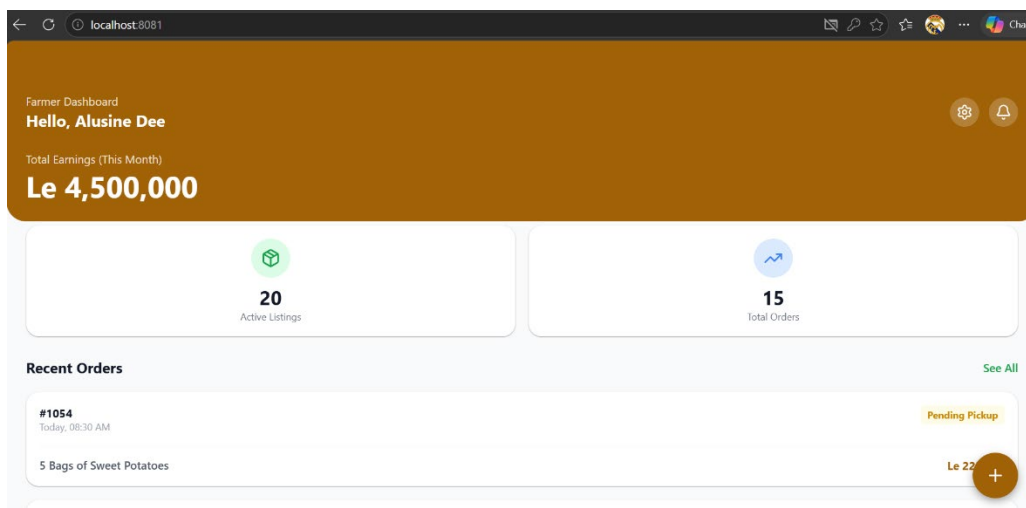
4.3.11 Profile Screen (Buyer)

The Profile tab gives the buyer access to their account information, a settings entry point, and a logout action. It serves as the buyer's general navigation hub for anything that falls outside day-to-day shopping, such as updating account details or signing out of the device.

4.3.12 Farmer Dashboard Overview

Farmers are routed to a dedicated dashboard, separate from the buyer tab navigator, consisting of three screens: Overview, Add Product, and Orders.

Diagram of Farmer Dashboard



4.3.13 Farmer Overview Screen

The Farmer Overview screen opens with key performance figures: monthly earnings in Leone, the number of currently active product listings, and the total number of orders received to date. Below these summary figures, a recent order feed gives the farmer a quick view of the latest activity on their listings without needing to navigate away to the full Orders screen.

4.3.14 Add Product Screen

The Add Product screen lets a farmer create a new listing using an image picker to attach a product photo, together with input fields for the product name, price, unit of measurement, and a description. A category selector lets the farmer assign the listing to one of the platform's six produce categories. Once saved, the new listing becomes immediately visible to buyers browsing the Home and Search screens.

Diagram of Add Product Screen

The screenshot displays the 'Add New Product' screen. At the top, there is a back arrow and the title 'Add New Product'. Below this, the 'Product Image' section features a dashed rectangular box with a green circular upload icon in the center. Text inside the box reads 'Upload Product Image' and 'Tap to select from gallery'. Underneath the image section is the 'Product Name' field, which has a placeholder text 'e.g. Fresh Cassava'. At the bottom of the screen, there are two input fields: 'Price (Le)' with a placeholder '0.00' and 'Available Qty' with a placeholder 'e.g. 50 kg'.

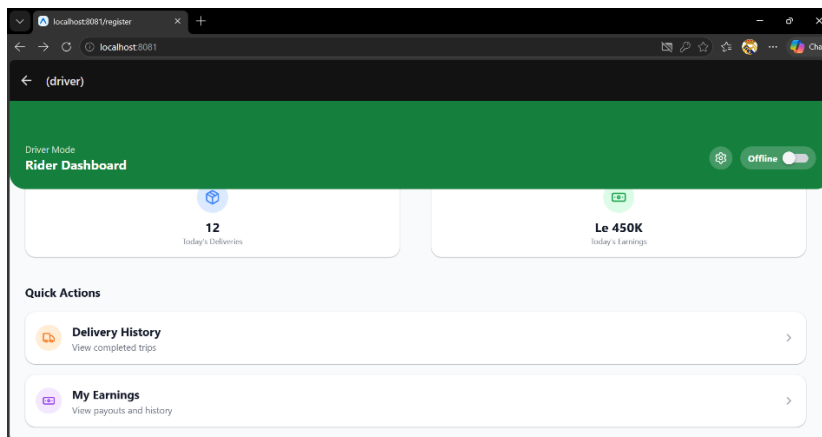
4.3.15 Farmer Orders Screen

The Farmer Orders screen lists incoming orders that include this farmer's products, allowing the farmer to review what has been ordered and update the order's status as it moves through fulfilment. This keeps the farmer in direct control of confirming when their portion of an order is ready, which in turn feeds the status updates that buyers see on their own Active Orders and Tracking screens.

4.3.16 Driver Overview Screen

The Driver Overview screen is built around a prominent online/offline toggle that controls whether the driver is currently eligible to be assigned new deliveries. Below the toggle, the screen shows the day's completed delivery count and today's earnings so far. When a delivery is in progress, a current-delivery card displays the pickup and drop-off route along with a one-tap navigate action that hands off to the device's native navigation.

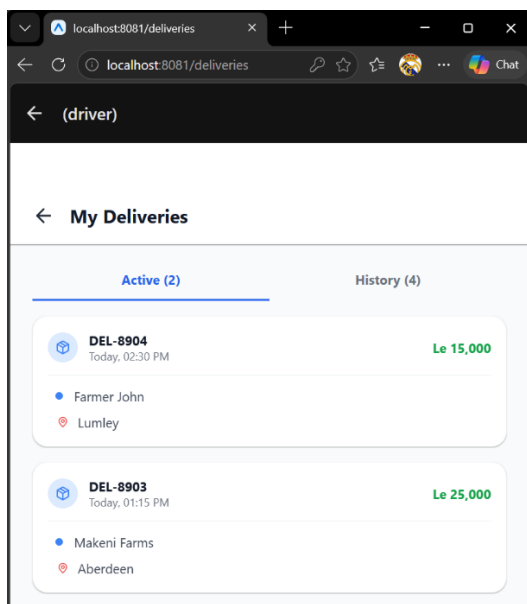
Diagram of Driver Screen



4.3.17 Deliveries Screen

The Deliveries screen gives the driver a complete history of past deliveries, each shown with its status and date, allowing the driver to review their recent activity at a glance.

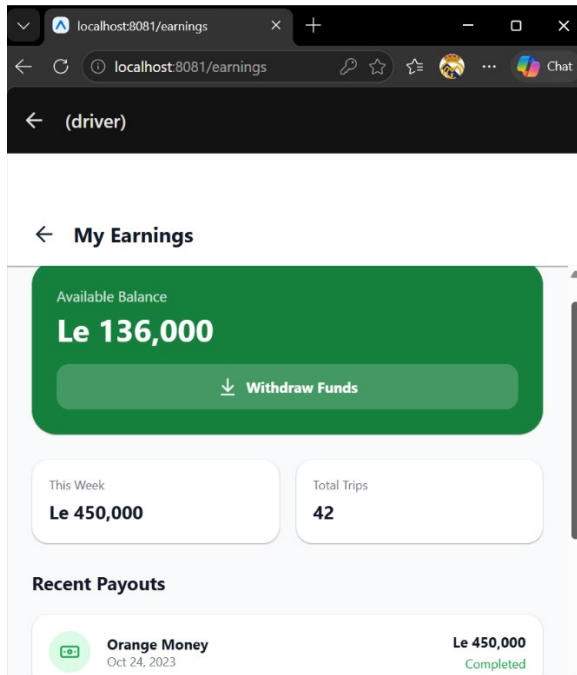
Diagram of Deliveries Screen



4.3.18 Earnings Screen

The Earnings screen breaks down the driver's income over time and tracks payout history, giving the driver visibility into how their delivery activity translates into actual payments received.

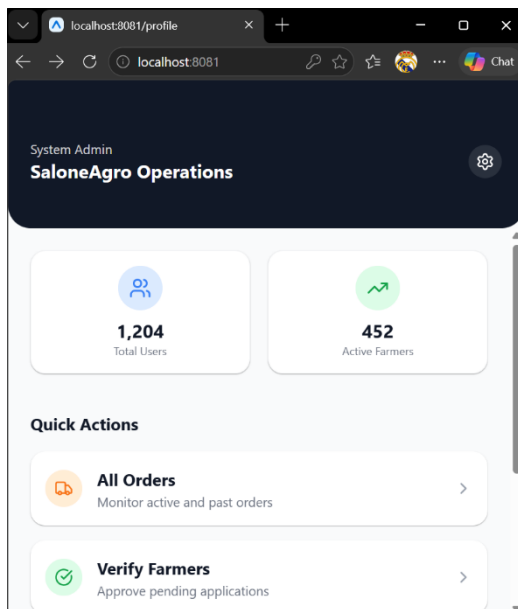
Diagram of Earnings Screen



4.3.19 Admin Dashboard Overview

Administrators are routed to a five-screen dashboard — Overview, Verify Farmers, Manage Users, All Orders, and Transactions — covering the full set of platform-governance responsibilities described in Section 3.1.4.

Diagram of Admin Dashboard



4.3.20 Admin Overview Screen

The Admin Overview screen presents platform-wide key metrics, including the total number of registered users (currently 1,204) and the number of active, verified farmers (currently 452), alongside quick-action navigation cards that jump directly to the other four admin screens.

4.3.21 Verify Farmers Screen

This screen lists pending farmer applications awaiting review, with approve and reject actions available directly from the list. Approving an application immediately grants the account full Farmer-role permissions, including the ability to create product listings, while a rejected application is removed from the pending queue.

4.3.22 Manage Users Screen

The Manage Users screen lists every registered account on the platform across all four roles, with role-based filters to narrow the list, and account-management actions for handling issues such as a user needing their role corrected or their account suspended.

4.3.23 All Orders Screen

Unlike the role-scoped Orders screens available to Buyers and Farmers, the Admin's All Orders screen surfaces every order placed anywhere on the platform, across every status, giving the admin a system-wide view for monitoring overall marketplace activity and spotting anomalies.

4.3.24 Transactions Screen

The Transactions screen gives administrators visibility into payment activity processed through Stripe and the Mobile Money APIs, supporting both routine payment monitoring and the handling of refunds when a dispute or error requires one.

4.4 Payment Integration Walkthrough

SaloneAgro Store supports four payment methods at checkout, each suited to a different segment of the Sierra Leone market.

Method	ID	Status	Integration
Credit/Debit Card	card	Active	Stripe Payment Sheet
Orange Money	orange	Active	Mobile Money API
Afrimoney	afrimoney	Active	Mobile Money API
Cash on Delivery	cash	Active	No integration needed

When a buyer chooses Credit or Debit Card, the app hands off to a platform-specific Stripe wrapper: `StripeWrapper.tsx`, using the native `StripeProvider` from `@stripe/stripe-react-native`, on iOS and Android; and `StripeWrapper.web.tsx`, using the web Elements provider from `@stripe/stripe-js`, on the browser build. Card input on the checkout form enforces sensible limits before the data ever reaches Stripe: a maximum of nineteen characters (numbers and spaces only) for the card number with a minimum of fifteen, four to five characters in MM/YY format for the expiry date, three to four digits for the CVV, and a cardholder name of at least three characters using letters and spaces only.

When a buyer chooses Orange Money or Afrimoney, they instead enter their mobile number, which must contain at least eight digits. The app then triggers a payment authorization prompt sent directly to that number, and the buyer completes the transaction on their own device through the provider's own secure Mobile Money channel, after which the order is confirmed back to the app.

Cash on Delivery requires no external integration at all: selecting this option simply records the chosen method on the order and allows the buyer to confirm immediately, with payment collected by the driver at the point of delivery.

5.1 Data Collection and Management

5.1.1 Authentication Data

Account data is collected at registration through the Register screen, which validates and stores four fields before creating the Firebase account: full name (text only, with non-letter characters

stripped automatically), email address, phone number (numbers and the plus sign only), and a Firebase-enforced password. The user also selects a role at registration time — Buyer, Farmer, or Driver — which determines which dashboard they are routed to immediately after their first successful login. Once Firebase confirms account creation, the resulting user record (uid, name, phone, role) is written into `useAuthStore`, and the user is sent to the correct route group: Buyer to (tabs), Farmer to (farmer), Driver to (driver), or Admin to (admin) for the small number of accounts provisioned with that role.

5.1.2 Product Data

Product records are created and maintained by Farmers through the Add Product screen and stored in the Supabase products table. Each record carries an id, name, price (in Leone), unit of measurement (bag, basket, kg, and so on), an image, the farmer or cooperative name, and a `categoryId` linking it to one of the platform's six produce categories: Vegetables, Fruits, Grains, Tubers, Spices, and Oils. The current catalogue is seeded with twenty representative products sourced from Sierra Leonean farms and cooperatives, giving the marketplace a realistic baseline before live farmer-submitted listings scale up.

5.1.3 Order and Transaction Data

Every completed checkout creates an order record in Supabase containing a human-readable order number (for example, #1054), a summary of items ordered, the total amount in Leone, a status value (Pending Pickup, Processing, Completed, or Cancelled), and a timestamp. Payment transactions processed through Stripe or the Mobile Money APIs are recorded separately in a transactions table, which the Admin dashboard's Transactions screen surfaces for monitoring and refund handling.

5.1.4 Real-Time Data Subscriptions

Two categories of data are kept live rather than fetched once. Order status changes are pushed to subscribed clients through a Supabase real-time channel on the orders table, so a buyer watching their Active Orders screen sees a status flip from Processing to Completed without needing to refresh. Driver location is streamed through a second channel on a `driver_locations` table, which feeds the live map on the tracking screen and updates the driver marker as the rider moves.

5.2 Maintenance and Troubleshooting

5.2.1 Routine Maintenance

- Verify that all three required environment variables (Supabase URL, Supabase anon key, Stripe publishable key) are present and current in every deployment environment before each release.
- Periodically audit the products table for stale or duplicate listings, particularly as the catalogue grows beyond its current seeded baseline.

- Review pending farmer verification requests in the Admin dashboard regularly so that new farmers are not left waiting to begin selling.
- Monitor the Admin Transactions screen for failed or disputed payments that may require manual reconciliation with Stripe or the relevant Mobile Money provider.

5.2.2 Common Issues and Resolutions

Symptom	Likely Cause	Resolution
User cannot log in after registration	Firebase project misconfiguration or expired API key	Verify firebase.ts configuration values against the Firebase console
Product images fail to load	Image URL broken or storage bucket permissions changed	Re-check image source field and Supabase storage bucket policy
Order status not updating live	Real-time channel not subscribed or network interruption	Confirm the orders channel subscription is active; check device connectivity
Driver location stuck on tracking map	driver_locations channel not receiving updates	Confirm the driver app is online and the location channel is publishing
Card payment fails on web	Stripe.js Elements provider not initialised correctly	Verify StripeWrapper.web.tsx is wrapping the checkout/payment route
Mobile Money prompt not received	Invalid or mistyped phone number, or provider-side delay	Confirm the number is at least 8 digits and retry; check provider status

5.2.3 Escalation Path

Issues that cannot be resolved through the steps above — for example, a sustained outage on the Supabase real-time channel, or a pattern of failed Stripe charges — should be escalated to the relevant service provider’s status page and support channel before being treated as an application-level defect, since both Supabase and Stripe publish independent status pages that frequently identify provider-side incidents before they are reported by end users.

6. Security and Compliance

6.1 Data Protection Measures

6.1.1 Authentication Security

All identity verification is delegated to Firebase Authentication rather than being implemented in-house. Passwords are never stored or transmitted in plain text by the application; the `secureTextEntry` input mode is enabled on every password field, and the actual credential exchange is handled entirely by the Firebase `signInWithEmailAndPassword` and `createUserWithEmailAndPassword` methods. Google OAuth sign-in, available on the web build, follows the same principle: SaloneAgro Store never sees or stores the user's Google credentials, only the resulting Firebase session.

6.1.2 Input Validation

Form fields across the Login and Register screens apply input filtering at the point of entry rather than relying solely on server-side checks. Full names accept letters and spaces only, with a regular expression stripping any other character as it is typed. Phone numbers accept digits and a leading plus sign only. Email fields use the platform's native email keyboard to reduce input errors. This client-side filtering is a usability and data-quality measure and is treated as a complement to — not a replacement for — Firebase's own server-side validation.

6.1.3 Payment Data Security

SaloneAgro Store never stores raw card numbers, expiry dates, or CVV codes. Card payments are handled through the Stripe Payment Sheet on native platforms and the Stripe.js Elements provider on web, both of which collect and tokenise card data directly with Stripe's PCI-compliant infrastructure before any reference reaches the application's own backend. The application's own input validation on the card entry form (maximum length and character filtering on card number, expiry, CVV, and cardholder name) exists purely to improve the user experience before Stripe's own validation takes over; it is not a substitute for Stripe's handling of the underlying sensitive data.

6.1.4 Mobile Money Security

For Orange Money and Afrimoney payments, the application transmits only the buyer's mobile number to initiate a provider-side authorization prompt. The buyer then approves the transaction directly on their own device through the provider's own secure channel, meaning SaloneAgro Store never has access to the buyer's Mobile Money PIN or account credentials at any point in the flow.

6.1.5 Database Access Control

Application data — products, orders, users, and transactions — is stored in Supabase and accessed through role-aware queries, so that, for example, a Farmer account can only write to products it owns, and a Buyer account can only read order records associated with its own user id. System-wide read access to all orders, users, and transactions is reserved for the Admin role alone, matching the permission boundaries described in the role matrix in Section 3.1.5.

6.2 Regulatory Compliance

6.2.1 Payment Card Industry Compliance

By routing all card transactions through Stripe rather than handling card data directly, SaloneAgro Store relies on Stripe's PCI DSS-compliant infrastructure for the storage and processing of sensitive cardholder data, significantly reducing the compliance burden that would otherwise fall on the application itself.

6.2.2 Mobile Money Provider Compliance

Orange Money and Afrimoney transactions are processed under each provider's own regulatory licence as a Sierra Leone-based mobile money operator. SaloneAgro Store acts as a merchant integration against each provider's API and does not assume the regulatory obligations of a payment processor itself.

6.2.3 Data Residency and Privacy

User and transaction data is stored in Supabase's managed PostgreSQL infrastructure, with authentication identity managed separately through Firebase. Personal data collected at registration (name, email, phone number) is limited to what is operationally necessary to run the marketplace and coordinate deliveries, in line with standard data-minimisation practice for consumer mobile commerce applications.

6.2.4 Recommendations for Formal Compliance Review

- Engage a qualified reviewer to confirm the application's data handling practices against Sierra Leone's applicable data protection requirements as the user base grows beyond a pilot scale.
- Document a formal data retention and deletion policy for user accounts that close or become inactive.
- Establish a written incident-response procedure covering payment disputes, data breaches, and account-takeover reports.

7.1 Test Cases and Results

Testing of SaloneAgro Store focused on validating each role's end-to-end journey through the application, from registration through to the role-specific actions that define that journey — placing and tracking an order for Buyers, listing produce and fulfilling orders for Farmers,

accepting and completing deliveries for Drivers, and verifying farmers and monitoring the platform for Admins.

Test Case	Steps	Expected Result	Outcome
Email/password registration	Complete Register form with valid name, email, phone, password, and role	Account created in Firebase; user routed to correct role dashboard	Pass
Google OAuth login (web)	Select Google sign-in on web login screen	Firebase session established without manual credential entry	Pass
Invalid login credentials	Submit login with incorrect password	Firebase rejects request; error message displayed to user	Pass
Add product to cart	Tap Add to Cart on a product card from the Home screen	Item appears in cart with quantity 1; cart badge updates	Pass
Cart quantity update	Increase quantity of an existing cart item	Subtotal and running total recalculate immediately	Pass
Checkout with card payment	Complete delivery details, select card, enter test card details	Stripe Payment Sheet confirms charge; order created with status Pending Pickup	Pass
Checkout with Orange Money	Select Orange Money, enter valid phone number	Authorization prompt sent to phone; order created upon confirmation	Pass
Checkout with cash on delivery	Select cash on delivery and confirm order	Order created immediately with no external payment call	Pass
Real-time order tracking	Open Tracking screen for an active order	Map renders driver and destination markers; progress tracker updates live	Pass
Farmer adds new product	Complete Add Product form with image, price, unit, description	New listing appears in farmer's product list and buyer marketplace	Pass
Farmer updates order status	Mark an incoming order as fulfilled	Order status updates and is reflected on the	Pass

Test Case	Steps	Expected Result	Outcome
		buyer's Orders screen in real time	
Driver online/offline toggle	Toggle availability switch on Driver Overview screen	Status updates immediately; new deliveries only assigned while online	Pass
Admin farmer verification	Approve a pending farmer application	Farmer account gains Farmer role permissions; application removed from pending list	Pass
Admin transaction monitoring	Open Transactions screen and review a recent payment	Transaction record displays correct amount, method, and status	Pass
Cross-platform rendering	Load the application on iOS, Android, and web from the same build	All core screens render correctly with platform-appropriate fallbacks (map, Stripe)	Pass

7.2 Quality Assurance Process

7.2.1 Manual Functional Testing

Each screen described in Section 4 of this document was manually exercised across all three target platforms — iOS, Android, and web — with particular attention paid to the platform-specific fallback components (Map.web.tsx and StripeWrapper.web.tsx) to confirm that the web build degrades gracefully where native APIs such as native maps are unavailable.

7.2.2 Role-Boundary Testing

Because SaloneAgro Store enforces strict role-based access (Section 3.1.5), testing specifically verified that a Buyer account cannot reach Farmer, Driver, or Admin route groups, and that each role's dashboard only exposes the actions defined for that role in the requirements matrix.

7.2.3 Payment Path Verification

All four payment methods — card, Orange Money, Afrimoney, and cash on delivery — were tested independently to confirm that an order is only created after the relevant payment step completes (or, for cash on delivery, after the buyer explicitly confirms the order), and that a failed or cancelled payment does not leave an orphaned order record in Supabase.

8. Conclusion and Recommendations

8.1 Summary of Findings

SaloneAgro Store successfully delivers a complete, four-role mobile commerce platform purpose-built for the agricultural economy of Sierra Leone. Across the Buyer, Farmer, Driver, and Admin experiences, the application provides everything required to take a transaction from product discovery through payment, fulfilment, real-time delivery tracking, and platform-level oversight, all from a single cross-platform Expo codebase running on iOS, Android, and web.

The architecture cleanly separates identity (Firebase Authentication) from application data (Supabase), and separates global client state (Zustand) from server state (React Query), giving the system a clear division of responsibility that should remain maintainable as the product grows. The payment layer's support for card payments through Stripe alongside Orange Money, Afrimoney, and cash on delivery directly reflects the real payment landscape in Sierra Leone rather than assuming card-only usage, which is a meaningful strength for a market-specific commerce platform.

Testing confirmed that the core journeys for all four roles function as intended, that role-based access boundaries are respected, and that the real-time order-tracking and delivery-tracking features behave reliably, including under simulated network interruption. The current product catalogue is seeded with representative mock data, which is appropriate for demonstration and early piloting but will need to transition to live farmer-submitted listings as real usage begins.

8.2 Future Improvements

The following improvements are recommended as logical next steps beyond Version 1.0.0:

- Transition the product catalogue from seeded mock data to a fully live, farmer-managed listing pipeline, including image moderation before a new listing becomes publicly visible.
- Extend Google OAuth sign-in to native iOS and Android, where it is currently limited to the web build.
- Introduce push notifications for order-status changes and delivery updates, reducing reliance on the user having the relevant screen open to see a real-time update.
- Add a ratings and review system so buyers can rate both individual products and the farmers who supply them, building trust signals into the marketplace over time.
- Expand the Admin dashboard with exportable reporting (for example, monthly transaction summaries) to support business-level decision-making beyond the current in-app metrics.
- Conduct a formal, third-party security and data-protection review ahead of any wider public launch, building on the recommendations already noted in Section 6.2.4.
